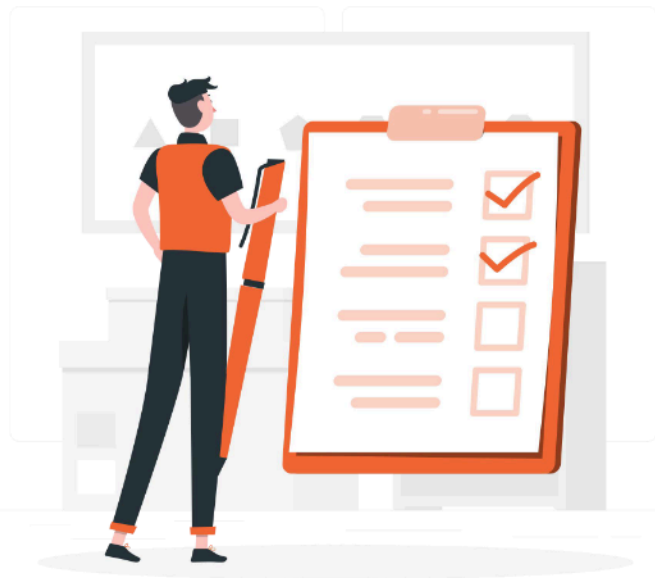




A/B Testing for LLMs

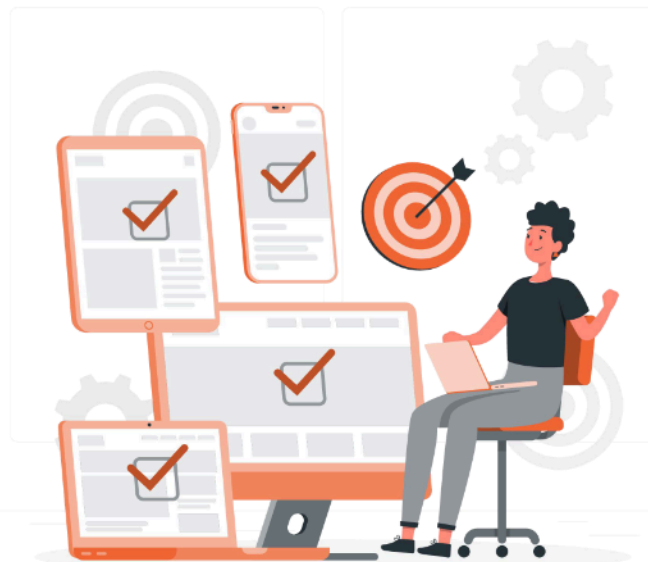
Lesson

1. Comparing prompts
2. Testing models (GPT-5.3, Gemini-3, DeepSeek)
3. Running evaluation experiments

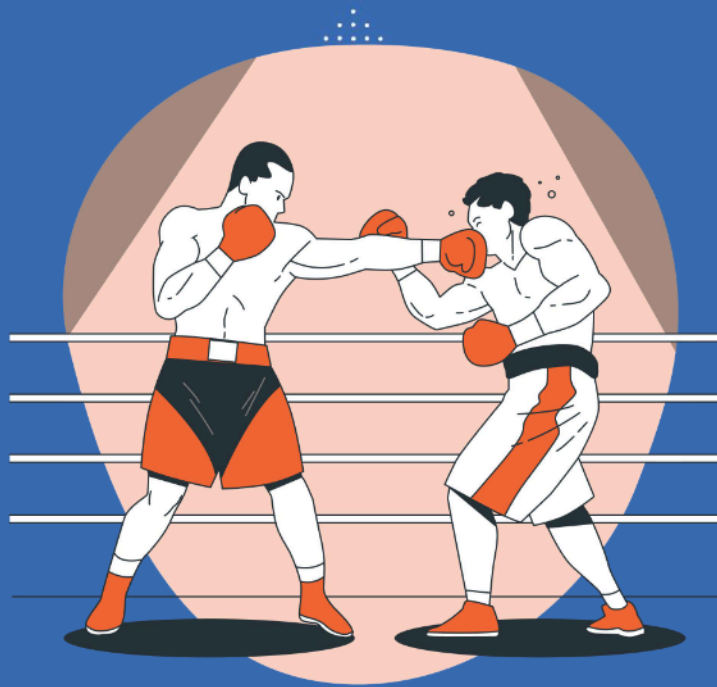


Learning Outcome

1. Understand LLM evaluation workflows and model comparison strategies
2. Design structured test sets for text and image evaluation
3. Apply LLM-as-a-Judge to evaluate and rank outputs
4. Run and analyze experiments to compare LLM performance

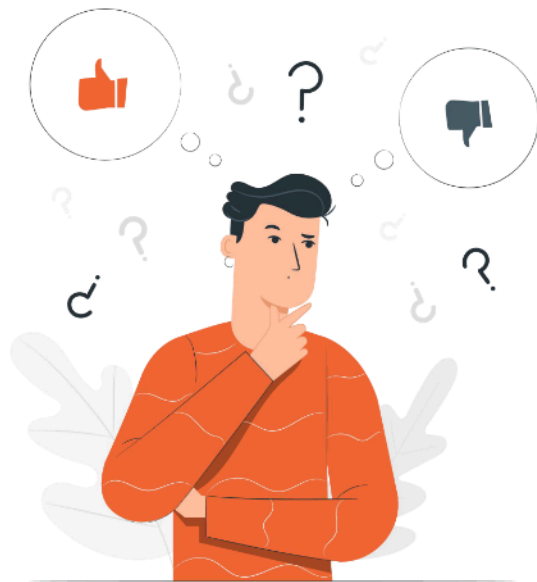


Comparing Prompts

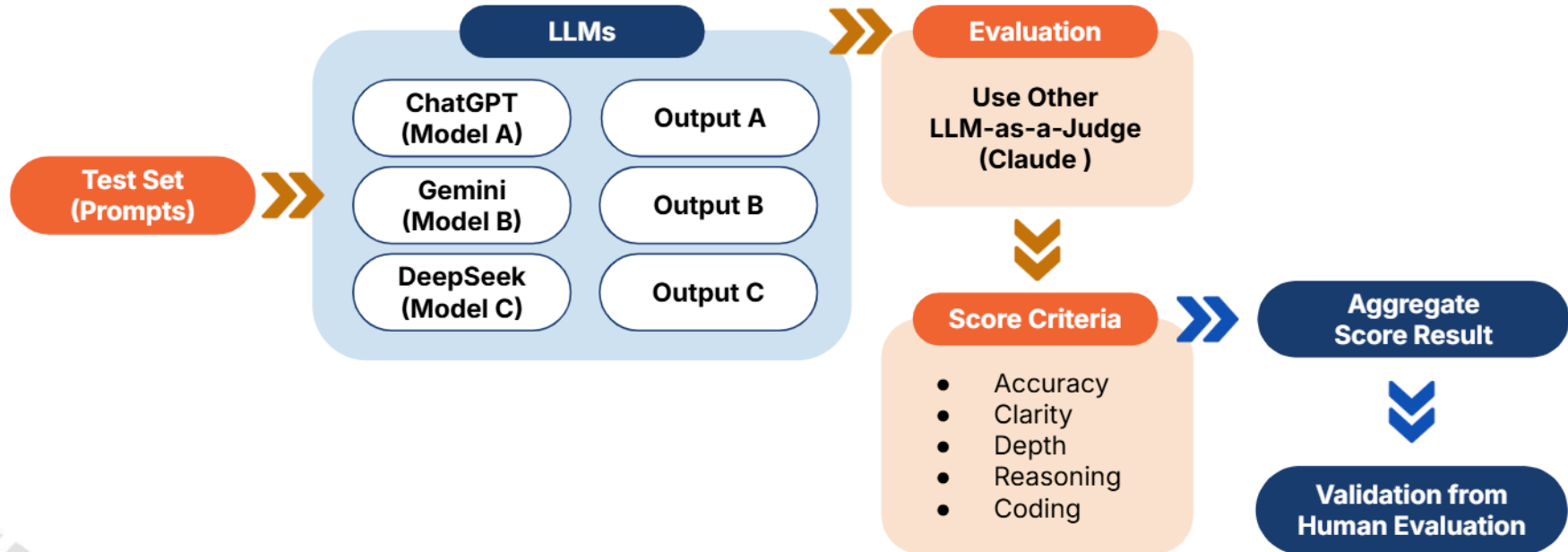


Comparing Prompts

When **comparing Large Language Models (LLMs)** (such as OpenAI models, Google models, or DeepSeek models), we use **several evaluation criteria**, like **accuracy, clarity, depth, reasoning, and coding**. These criteria help determine which model performs better for a given task.



Flow Diagram for Comparing LLMs



Testing Models





Text Generation Prompts

Test Set (Prompts)



Goal	Prompt
Instruction following, software testing knowledge, and output structure.	You are a QA analyst validating a banking app's login API. Generate 10 test cases covering positive, negative and boundary conditions. Include inputs, expected outputs and validation notes in a table.
Identify unclear or incomplete parts of a requirement and propose questions or acceptance criteria.	You are a senior QA consultant reviewing an e-commerce user story. Story: "When users add products to the basket, the system should automatically calculate discounts and display updated totals in real time." Identify potential ambiguities or missing acceptance criteria. Output as 'Observation → Clarification Needed' pairs. Break the requirement into atomic, testable statements. Classify each as Functional, Non-Functional, or UI-related. List five clarification questions a QA would raise about discount logic, triggers, or thresholds.
Encourage the AI to think like a user to discover hidden bugs.	Suggest 20 exploratory testing ideas for an e-commerce checkout flow. Cover payment methods, promo codes, cart manipulation, and network delays. For each idea, describe what to explore, what to observe, and why it's important.



Image Generation Prompts

Test Set (Prompts)

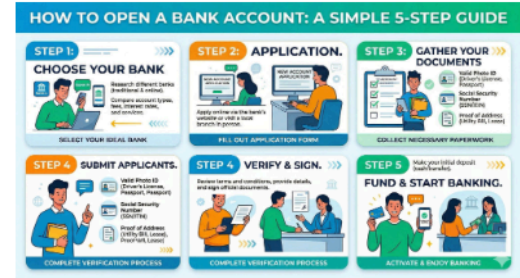
Prompts

A 5 panel instructional page on how to open bank account.

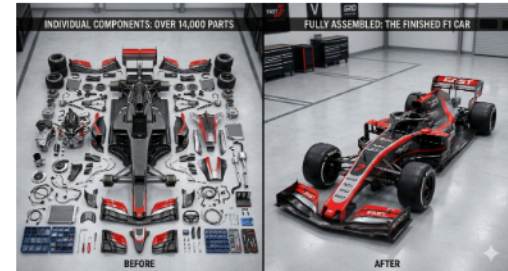
ChatGPT



Gemini (Nano Banana)



A before and after image of a F1 car, with the left half being the individual pieces and the right half being fully constructed.



Test Set (Prompts)

Prompts

A full board top down image of a legal chess game in the mid game where a white bishop is checking the black king.

Generate an image divided into 4 panels, showing the exterior of the same monorail station from the different viewing angles of top, front, back and side.

ChatGPT



Gemini (Nano Banana)





LLM-as-a-Judge

Claude LLM-as-a-Judge

What is LLM-as-a-Judge?

1. Uses **one LLM to evaluate** outputs from other LLMs
2. Scores responses based on predefined criteria
3. Helps compare models faster and more consistently

Why Use Claude?

1. Strong at structured reasoning and explanation
2. Good at comparing responses with rubric-based scoring
3. Produces clear evaluation notes, not just scores
4. Suitable as an automated first-pass judge

Prompt to Evaluate LLM Results

1. Give Claude the same prompt and all model outputs
2. Hide model names as Response A, B, and C
3. Ask Claude to score each response by criteria
4. Request final ranking and short justification

Running Evaluation Experiments





Text Generation Evaluation

Result from Prompt 1

Prompt

You are a QA analyst validating a banking app's login API. Generate 10 test cases covering positive, negative and boundary conditions. Include inputs, expected outputs and validation notes in a table.

Below are 10 QA test cases for validating a Banking App Login API. They include positive, negative, and boundary scenarios. Assumptions:

- Endpoint: `POST /api/v1/auth/login`
- Required fields: `username`, `password`
- Username length: 3-50 chars
- Password length: 8-64 chars
- Response success: 200 + auth token
- Invalid credentials: 401
- Validation errors: 400

TC ID	Scenario	Input (username / password)	Preconditions
TC-01	Valid login (standard positive case)	<code>john_doe / SecurePass123!</code>	User exists and account active
TC-02	Valid login with minimum username length	<code>abc / SecurePass123!</code>	User exists with username length = 3
TC-03	Valid login with maximum allowed password length	<code>john_doe / 64-char valid password</code>	User exists with 64-char password
TC-04	Incorrect password	<code>john_doe / WrongPass123!</code>	User exists
TC-05	Non-existent username	<code>unknown_user / SecurePass123!</code>	User not in DB
TC-06	Missing password field	<code>john_doe / </code>	None
TC-07	Empty username and password	<code> / </code>	None
TC-08	Username exceeds maximum length	<code>51-character username / SecurePass123!</code>	None
TC-09	SQL injection attempt in username	<code>admin' OR '1'='1 / password123</code>	None
TC-10	Locked account login attempt	<code>locked_user / SecurePass123!</code>	Account status = locked after failed attempts

Additional validation checks (recommended for all cases):

- Response `content-type` = `application/json`
- No sensitive data (password, hash) returned
- Rate limiting headers present if implemented
- Audit logs created for login attempts
- HTTPS required and tokens securely generated (JWT structure if applicable)

GPT5.3

As a QA analyst, my goal is to ensure this API is not only functional but also resilient against edge cases and common security vulnerabilities. Below is a comprehensive test suite for the `/login` endpoint.

Banking App Login API Test Cases

ID	Test Type	Input (JSON Payload)	Expected Output (HTTP & Body)
01	Positive	Valid <code>username</code> & <code>password</code>	200 OK + Auth Tok
02	Negative	Valid <code>username</code> + Incorrect <code>password</code>	401 Unauthorized
03	Negative	Non-existent <code>username</code>	401 Unauthorized
10	Negative	Special characters in <code>password</code>	200 OK (if valid)

Export to Sheets

Key Observations for Banking Security

- Rate Limiting:** Beyond individual account lockouts, I'd recommend testing if the API throttles multiple requests from the same IP to prevent brute-force attacks.
- Data Masking:** Ensure that the `password` is never returned in the response body or logged in plain text in the server-side logs.
- MFA Trigger:** For a banking app, a successful login (200 OK) might actually return a partial success state requiring a second-factor (OTP) before a full session token is issued.

Gemini

Here is a comprehensive set of 10 test cases for a banking app login API, covering positive, negative, and boundary scenarios.

Assumptions for this API:

- Endpoint: `/api/v1/auth/login`
- Method: `POST`
- Content-Type: `application/json`
- Request Body:

```
json
{
  "username": "user@example.com or userID",
  "password": "password123!"
}
```

Business Rules:

- Username is typically an email address or a customer ID (min 5, max 50 characters).
- Password is case-sensitive (min 8, max 30 characters).
- After 3 failed attempts, the account is locked for 15 minutes.

Login API Test Cases

ID	Test Case Title	Inputs (Username / Password)
TC_001	Positive - Valid Credentials	<code>john.doe@bank.com / ValidPass@123</code>
TC_002	Negative - Incorrect Password	<code>john.doe@bank.com / WrongPass@456</code>
TC_003	Negative - Non-Existent Username	<code>fake.user@bank.com / AnyPass@123</code>

DeepSeek

Score Result from Prompt 1



A

Criterion	Score	Notes
Accuracy	8/10	All 10 cases are technically correct. HTTP codes are appropriate. TC-05 correctly notes response should be identical to wrong password (anti-enumeration). SQL injection properly handled.
Clarity	8/10	The standout structural advantage is the Preconditions column — critical for test execution and often missed. Table is clean and readable.
Depth	8/10	Covers both min and max boundary cases for username/password separately (TC-02, TC-03, TC-08). Additional validation checklist at the bottom (JWT structure, audit logs, rate limiting) adds solid depth.

B

Criterion	Score	Notes
Accuracy	8/10	Mostly accurate. Minor flag: 423 Locked (RFC 4918, WebDAV origin) is non-standard for banking APIs — 403 is more widely accepted. TC-09 (Malformed JSON) is a unique and accurate addition the others miss.
Clarity	7/10	Table is clean but lacks concrete input examples (just descriptions, not actual values). "Input (JSON Payload)" column header implies JSON but shows no JSON.
Depth	9/10	Best depth of the three. Timing attack mention in TC-03 ("response time consistent to prevent timing attacks") is a senior-level security catch. MFA partial success state and malformed JSON are unique, high-value additions not in A or C.

C

Criterion	Score	Notes
Accuracy	9/10	Most accurate overall. Full JSON body in expected outputs (not just status codes). TC-006/TC-007 correctly state "depends on business rules" — a nuanced and accurate acknowledgment. TC-010 step-by-step lockout flow is precisely mapped.
Clarity	10/10	Best formatted. Real example values (john.doe@bank.com , ValIdPass@123), full JSON response bodies in Expected Output, and the multi-step TC-010 are all immediately usable by an automation engineer. Assumptions section is the most complete.
Depth	8/10	Solid coverage but misses B's advanced security observations (timing attacks, MFA trigger, malformed JSON, special char hash handling).

Criterion	A	B	C
Accuracy	8	8	9
Clarity	8	7	10
Depth	8	9	8
Total (avg)	8	8	9

Ranking: C > A = B

- C wins for usability and clarity — its expected outputs with real JSON bodies make it the most valuable for a team.
- B is the depth champion with unique cases (malformed JSON, timing attack, MFA state) that show the most security expertise.
- A is the most execution-ready due to the Preconditions column, which B and C both lack.

Ideal answer would combine: C's structure + B's advanced security observations + A's Preconditions column.

Result from Prompt 2



Prompt

- You are a senior QA consultant reviewing an e-commerce user story.
- Story: "When users add products to the basket, the system should automatically calculate discounts and display updated totals in real time." Identify potential ambiguities or missing acceptance criteria. Output as 'Observation → Clarification Needed' pairs. Break the requirement into atomic, testable statements. Classify each as Functional, Non-Functional, or UI-related.
- List five clarification questions a QA would raise about discount logic, triggers, or thresholds.

1. Potential Ambiguities / Missing Acceptance Criteria

Observation → Clarification Needed

- "Automatically calculate discounts" → What types of discounts are supported (percentage, fixed amount, coupon, tiered, buy-X-get-Y)?
- "Display updated totals" → Which totals should be shown (subtotal, discount amount, tax, shipping, final payable)?
- "Real time" → What is the acceptable update latency (instant UI update, <1s, <3s)?
- Discount scope unclear → Are discounts applied per product, per category, or at basket level?
- Multiple discounts scenario → Can multiple discounts stack or should only the highest discount apply?
- Eligibility rules undefined → Are discounts dependent on quantity, user role, membership tier, location, or time-based campaigns?
- Trigger events unspecified → Should recalculation happen when item quantity changes, item removal, coupon entry, or product replacement?
- Rounding rules missing → How should rounding be handled for currency values after discount calculations?
- Tax calculation order unclear → Are taxes applied before or after discounts?
- UI feedback unspecified → Should users see which discount was applied and how it affected the total?

2. Atomic, Testable Statements

ID	Atomic Requirement	Classification	
R1	When a user adds a product to the basket, the system recalculates the basket total.	Functional	
R2	The system determines if the added product qualifies for any discount rule.	Functional	
R3	If a discount rule is applicable, the system calculates the discount amount correctly.	Functional	
R4	The system updates the basket subtotal after adding an item.	Functional	
R5	The system displays the calculated discount amount in the basket summary.	UI	

GPT5.3

As a senior QA consultant, my initial assessment is that this user story is in early development. It lacks the edge-case definitions required to prevent logical flow—where financial discrepancies directly impact customer trust.

Review: Observations & Clarifications

Observation

"Automatically calculate"

Clarification Needed

Does this include manual coupon entry, or only system-wide promotions (e.g., "Buy 1 Get 1")?

Atomic Testable Statements

To ensure high test coverage, I've broken the story into granular requirements:

- Functional:** The system must validate the eligibility of a product for an active promotion upon the 'Add to Basket' action.
- Functional:** If a discount threshold (e.g., "Spend \$100") is met, the discount must be applied to the subtotal immediately.
- Functional:** If an item is removed from the basket and the total falls below a threshold, the discount must be revoked.
- Non-Functional:** The "real-time" calculation must complete and update the UI within 200ms of the basket modification.
- UI-related:** The basket must display a "Savings" line item showing the total amount deducted from the original price.
- UI-related:** A visual indicator (e.g., a loading spinner or highlight) should appear briefly while the new total is being fetched.

Gemini

Here is a structured QA review of the user story, breaking down the requirements into necessary clarifications.

Part 1: Ambiguities & Missing Criteria (Observation → Clarification)

Observation (The Ambiguity)	Clarification Needed (The Question)	Classification
The story mentions "automatically calculate discounts" but does not specify what triggers the discount. Is it a coupon code entry, a buy-one-get-one (BOGO) offer, a volume-based tier (e.g., spend \$100 get 10% off), or a loyalty program?	Trigger Definition: What specific actions or conditions cause a discount to be applied (e.g., entering a promo code, automatic volume discount, user group status)?	Functional
The term "updated totals" is vague. Does it refer only to the subtotal, or does it include tax, shipping estimates, and the final grand total?	Total Scope: Which specific fields on the page need to be updated? (e.g., Subtotal, Discount Line Item, Tax, Shipping, Grand Total).	Functional
The phrase "in real time" is an ambiguous non-functional requirement. Does this mean after a page refresh, after the user clicks "Update," or as they are typing a coupon code (debounce)?	Real-time Definition: What is the specific trigger for the recalculation? (e.g., On blur from coupon field, on click of "Apply," on quantity change, or on selection of a product variant).	Non-Functional

Part 2: Atomic & Testable Statements

Here is the user story broken down into specific scenarios that can be written as test cases.

- [Functional] If a user adds a product that is part of a "Buy 1 Get 1 Free" promotion to the basket, the system shall add the second unit at \$0.00 cost.
- [Functional] If a user enters a valid coupon code "SAVE10" in the designated field, the system shall deduct 10% from the subtotal before tax.
- [Functional] If a user applies two discount codes, the system shall apply only the one with the higher value, as per the "best discount only" business rule.

DeepSeek

Score Result from Prompt 2

Criterion	A	B	C
Accuracy	8	9	9
Clarity	8	7	10
Depth	8	9	9
Reasoning	8	9	9

Ranking: C > B > A

- C wins on usability and senior-level scope (concurrency, regional logic)
- B wins on domain reasoning with the BOGO/inventory edge case and tax sequencing
- A wins on breadth (most observations and atomic requirements) but lacks specificity

Best combined answer = A's breadth + B's domain reasoning + C's structure and concrete values

A

Criterion	Score	Notes
Accuracy	8/10	All 12 atomic requirements (R1–R12) are well-formed and accurately classified (Functional/UI/Non-Functional). Observations are relevant and correctly identified.
Clarity	8/10	Clean three-part structure. The tabular format for atomic requirements with ID + Classification is the most scannable of the three. Easy to hand off to a dev.
Depth	8/10	Best breadth of observations (10 items vs B's 4 and C's 5). Rounding rules, tax order, and eligibility rules are all present. Most atomic statements (12) of any answer.
Reasoning	8/10	Logical flow: ambiguity → requirement → clarification. Threshold logic question (what if user drops below discount threshold?) shows good negative-path thinking.
Coding	5/10	Offers to show a test suite but doesn't. No concrete values, no Gherkin, no pseudo-specs. Atomic statements are functional but generic.

B

Criterion	Score	Notes
Accuracy	9/10	Highly accurate. 200ms NFR is specific and testable. Tax sequencing flagged with "legal/accounting implications" — accurate and important. BOGO + out-of-stock edge case is a real production scenario.
Clarity	7/10	Observations table is compact (only 4 rows). Atomic statements are bullets without IDs or table structure — harder to reference in sprint planning.
Depth	9/10	Inventory/BOGO edge case is the most unique observation across all answers. WebSocket implication for "real-time," visual loading indicator for UX — these show system-level and frontend awareness.
Reasoning	9/10	Strong opening context ("too thin for development, financial discrepancies impact customer trust"). The inventory/BOGO case and tax sequencing demonstrate business domain reasoning, not just QA thinking.
Coding	6/10	Offers Gherkin but doesn't write it. The 200ms threshold and "loading spinner" mention are more implementation-specific than A, but still no specs or code.

C

Criterion	Score	Notes
Accuracy	9/10	All classifications are correct. Debounce reference for real-time coupon validation is technically precise. Specific values (<code>SAVE10</code> , <code>FAKECODE</code> , <code>\$100.00</code> , <code>500ms</code>) are accurate and testable.
Clarity	10/10	Best structured answer. Three labeled parts, markdown tables with classification column, meeting-ready question phrasing. Immediately usable as sprint documentation.
Depth	9/10	Concurrency (dual-tab basket sync), regional differences (USD vs EUR promo), product exclusions (gift cards), user tiers (Gold members) — all senior-level additions not in A or B.
Reasoning	9/10	Questions framed as if spoken in a backlog refinement meeting — shows process maturity. Error handling scenario (backend validation fail → what does UI do?) shows end-to-end thinking.
Coding	7/10	Atomic statements include specific values and exact behaviours (e.g., "system shall add the second unit at \$0.00 cost") — directly convertible to test scripts. Closest to executable specs without being actual Gherkin.

Result from Prompt 3

Prompt

Suggest 20 exploratory testing ideas for an e-commerce checkout flow. Cover payment methods, promo codes, cart manipulation, and network delays. For each idea, describe what to explore, what to observe, and why it's important.

Below are 20 exploratory testing ideas for an e-commerce checkout flow, covering payment methods, promo codes, cart manipulation, and network delays. Each idea includes what to explore, what to observe, and why it matters.

#	What to Explore	What to Observe	Why It's Important
1	Add multiple items to the cart and proceed to checkout	Cart totals, tax, shipping, and final price calculation	Ensures correct price aggregation and prevents billing errors
2	Change item quantities during checkout	Real-time recalculation of totals and discounts	Detects issues with dynamic pricing updates
3	Remove items from cart at checkout page	Updated totals, removal confirmation, discount re-evaluation	Ensures system handles cart changes properly
4	Apply a valid promo code	Discount application, UI feedback, final price update	Confirms correct promo logic and user feedback
5	Apply an expired promo code	Error message clarity and checkout continuity	Prevents incorrect discount application
6	Try stacking multiple promo codes	System behavior (stacking vs rejecting additional codes)	Verifies discount rules enforcement
7	Apply promo code and then modify cart contents	Discount recalculation based on updated cart value	Ensures promo eligibility logic works dynamically
8	Enter invalid promo code formats	Validation messages and input restrictions	Prevents injection or malformed input issues
9	Test checkout with credit/debit card payment	Payment authorization, confirmation page, order creation flow	Validates the most common payment flow
10	Test checkout with digital wallet (e.g., Apple Pay/Google Pay equivalent)	Redirect handling, payment confirmation, order status	Ensures wallet integrations work correctly

GPT5.3

To truly stress-test an e-commerce checkout, we need to move beyond automation and think like a frustrated, distracted, or even slightly malicious user.

Below are 20 exploratory testing ideas categorized by their potential to break the logic of your checkout flow.

Payment Methods & Security

#	What to Explore	What to Observe	Why It's Important
1	Expired Cards	Does the UI catch the expiry date before the API call?	Reduces server load and improves UX by preventing unnecessary processing.
2	Insufficient Funds	Does the system offer a clear "Retry with different method" option?	Prevents cart abandonment by keeping the user in checkout loop.
3	Clipboard XSS	Paste a script tag into the "Cardholder Name" field.	Essential for preventing Cross-Site Scripting (XSS) via stored payment data.
4	3D Secure Timeout	Initiate a 3D Secure check but let the timer run out.	Ensures the session doesn't "hang" or incorrectly mark the order as paid.
5	Multi-Tab Payment	Open two tabs of the same checkout; pay in one, then try to pay in the other.	Prevents double-charging or duplicate order generation.

Gemini

Here are 20 exploratory testing ideas for an e-commerce checkout flow. Each entry follows the structure: **What to Explore?** (The action), **What to Observe?** (The result), and **Why It's Important?** (The risk).

Payment Methods

1. The "Partial Payment" Illusion

- Explore:** Initiate a payment with a low-balance gift card combined with a credit card. Complete the first payment method, but abandon the process or close the browser immediately after the secondary payment is requested.
- Observe:** Check if the gift card balance is still deducted even though the final order confirmation was never received. Look for inventory holds that aren't released.
- Why:** Prevents financial discrepancies where customer balances are consumed without successful order completion.

2. Payment Gateway Timeout Simulation

- Explore:** Use a proxy tool (like Charles or Fiddler) to simulate a slow network or break the connection to the payment gateway provider after clicking "Pay Now," but before receiving a success/failure response.
- Observe:** Does the system show a generic "Something went wrong" message, or does it enter a "Processing" state? Check if the user is charged despite the error.
- Why:** Uncovers "inconsistent state" bugs where the order is stuck between pending and paid, leading to customer service nightmares.

3. Saved Credit Card Expiry Date Manipulation

- Explore:** Add a saved card to a user account. Manually modify the browser's local data or the database to set the card's expiry date to last month. Use this card at checkout.
- Observe:** Does the front-end validation catch this before submission, or does it get sent to the gateway and fail? How is the error presented to the user?
- Why:** Stored payment details can change over time; the system must re-validate data at the point of transaction, not just when the card was saved.

DeepSeek

Score Result from Prompt 3

A

Criterion	Score	Notes
Accuracy	8/10	All 20 ideas are valid and relevant. Coverage of payment methods, promos, cart, and network is balanced. No incorrect assumptions.
Clarity	8/10	Clean single-table format is easy to scan. Consistent 3-column structure (Explore / Observe / Why) throughout.
Depth	6/10	Ideas are broad but shallow — each observation is one sentence. No specific tooling, no mention of race conditions, XSS, or backend-level exploits.
Reasoning	7/10	"Why It's Important" column is present but generic (e.g., "Prevents invalid order creation"). Lacks business impact specificity.
Coding	4/10	No tools, no techniques, no specific manipulation methods mentioned.

B

Criterion	Score	Notes
Accuracy	9/10	All ideas are accurate. Clipboard XSS, 3D Secure timeout, and guest-to-user cart merge are legitimate and well-targeted test scenarios.
Clarity	8/10	Excellent use of emoji-categorized sections (🔍 🛒 🛡️). One table per category improves scanability vs. Answer A's monolithic table.
Depth	9/10	Significantly more advanced: XSS via clipboard, 3DS timeout, case-sensitivity for promo codes, negative quantity via browser console, out-of-stock race condition.
Reasoning	9/10	Business impact is more specific — "Prevents cart abandonment", "infinite discounts where the store owes the customer money", "conversion-focused" reasoning.
Coding	5/10	Implies browser console manipulation and interceptor/proxy usage but doesn't name specific tools.

C

Criterion	Score	Notes
Accuracy	10/10	All 20 scenarios are technically precise. The API replay attack (#7), AVS fraud detection (#4), rounding exploitation (#9), and DOM state leak (#5) are all real-world vulnerabilities.
Clarity	8/10	Narrative paragraph format is rich but harder to scan quickly than a table. Each entry is self-contained and detailed — better for a test plan doc than a sprint card.
Depth	10/10	Deepest by far. Includes: partial payment abandonment with gift cards, Charles/Fiddler proxy simulation, CDN/font failure blocking, hosts file manipulation, database expiry modification, Postman API replay, penny rounding exploitation.
Reasoning	10/10	Outstanding specificity — "penny discrepancies accumulate to significant financial losses over thousands of transactions", "negative revenue orders", "customer service nightmares", "AVS address verification", "legal/fulfillment chargebacks".
Coding	7/10	Explicitly names Charles, Fiddler, Postman, browser dev tools (throttling, DOM inspection), hosts file blocking, and database-level manipulation. The most technically detailed of the three.

Criterion	A	B	C
Accuracy	8	9	10
Clarity	8	8	8
Depth	6	9	10
Reasoning	7	9	10

Ranking: C > B >> A

- C dominates on depth and reasoning — the only answer that reads like it was written by someone who has actually broken production checkout systems. The specific tools and precise business impact language set it apart.
- B is strong and more scannable — ideal for a team standup or quick test planning session.
- A is a solid starting framework but lacks the specificity needed to uncover real-world bugs.



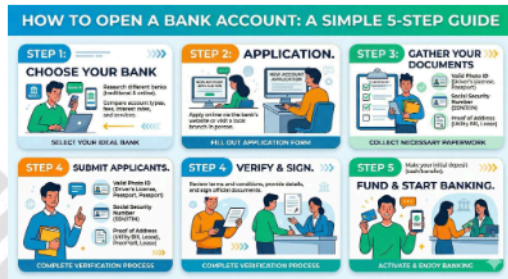
Image Generation Evaluation

Test Set (Prompts)

ChatGPT



Gemini (Nano Banana)



Claude LLM-as-a-Judge

Criterion	Score	Explanation
Prompt Alignment	10/10	Perfectly matches the prompt — exactly 5 numbered panels covering a complete bank account opening flow, with a clear title and congratulatory footer.
Visual Quality	9/10	Warm, polished illustrated style with consistent color palette, clean typography, and well-rendered scene details throughout all panels.
Realism	7/10	Stylized illustration rather than photorealistic, but the depicted scenarios (browsing, visiting bank, filling forms) feel believable and grounded.
Composition	9/10	Well-balanced 2-2-1 grid layout with a header, numbered step indicators, clear descriptions, and a closing footer banner — flows naturally top to bottom.
Artifact Errors	8/10	Minor AI text artifacts in Panel 2 (document text is slightly garbled), and the laptop screen in Panel 1 contains blurry placeholder content.

Final Ranking

Rank	Image	Avg Score	Notes
1st	Image 1 (Illustrated)	8.6	Strong prompt alignment, polished style, minor text artifacts
2nd	Image 2 (Refused)	0	Failed to generate — refusal was arguably unwarranted

Image 1 takes the win. Image 2's fatal flaws were the panel count mismatch and the artifact errors, both hard to overlook in an instructional design context.

Criterion	Score	Explanation
Prompt Alignment	5/10	Prompt says 5 panels, but image has 6.
Visual Quality	8/10	Clean flat illustration style, consistent palette.
Realism	7/10	Believable scenes, coherent character style throughout.
Composition	7/10	Balanced 3x2 grid, clear headers and footers per panel.
Artifact Errors	4/10	"Step 4" appears twice, garbled text ("Proofdrft"), and "Submit Applicants" is wrong.

Test Set (Prompts)

ChatGPT



Gemini (Nano Banana)



Claude LLM-as-a-Judge

Criterion	Score	Explanation
Prompt Alignment	6/10	4 views exist but are mislabeled — the aerial top panel is labeled "FRONT," "FRONT" appears twice, and no panel is actually labeled "TOP," missing one required angle explicitly.
Visual Quality	9/10	Strikingly beautiful photorealistic 3D render with rich warm lighting, detailed environment, and lush greenery.
Realism	7/10	High-quality CGI render that is convincing at a glance, but clearly synthetic — perfect surfaces and unrealistically clean surroundings break immersion.
Composition	7/10	The asymmetric layout (1 large top + 2 middle + 1 bottom) is visually dynamic but inconsistent — the panels aren't equal in size or perspective angle.
Artifact Errors	7/10	Signage text across all panels is garbled and unreadable (a common AI text generation failure), and some edge geometry looks slightly warped.

Criterion	Score	Explanation
Prompt Alignment	8/10	All 4 panels are correctly labeled (TOP VIEW, FRONT VIEW, BACK VIEW, SIDE VIEW), but the panels appear to show architecturally different stations, violating the "same station" requirement.
Visual Quality	8/10	Photographic sharpness and natural lighting across all panels, though the top-view panel's large text overlay feels amateurish.
Realism	9/10	Sourced from real-world photography making it maximally realistic, though the composite nature undermines a unified sense of place.
Composition	8/10	Clean, symmetrical 2x2 grid is easy to read and well-balanced, with consistent label placement across all quadrants.
Artifact Errors	7/10	Diamond watermark in bottom-right corner, and the heavy blue text overlay in the TOP VIEW panel feels digitally pasted rather than integrated.

Final Ranking

Rank	Image	Avg Score	Strength	Critical Weakness
1st	Image 2 (Photo)	8	Correct 4-panel structure, maximum realism	Panels show different stations, not the same one
2nd	Image 1 (Render)	7.2	Visually stunning, consistent station design	Mislabeled views — no true "TOP" panel shown

Key Insight: **Image 2 wins on structure and realism while Image 1 wins on visual consistency** (same station shown throughout). Neither fully satisfies the prompt — the ideal image would combine Image 1's consistent station design with Image 2's correct 4-panel labeling.

Test Set (Prompts)



ChatGPT



Gemini (Nano Banana)



Claude LLM-as-a-Judge

Criterion	Score	Explanation
Prompt Alignment	5/10	Top-down full board is present, but the red highlight marks the white piece (not the black king in check), making the check relationship ambiguous and the board position appears more like early-game than midgame.
Visual Quality	9/10	Professional-grade photography with warm lighting, sharp focus, and rich depth — visually stunning.
Realism	10/10	This is a photograph of real physical pieces on a real board — maximum realism.
Composition	9/10	Excellent environmental storytelling (coffee, notebook, clock, captured pieces bowl) creates a natural, balanced scene.
Artifact Errors	7/10	Several pieces along the bottom edge appear merged or malformed, and the board position contains some geometrically odd-looking pieces.

Criterion	Score	Explanation
Prompt Alignment	9/10	Precisely matches the prompt — white bishop on e5 with a yellow arrow pointing to the black king on g7 (highlighted in red), in a clear midgame position.
Visual Quality	7/10	Clean and crisp 2D interface, but flat by nature — no lighting, texture, or depth.
Realism	4/10	A digital UI representation, not a photographic or 3D rendering — inherently abstract.
Composition	8/10	The arrow and red king highlight effectively direct focus to the check relationship; coordinates add helpful context.
Artifact Errors	8/10	Minor watermark diamond in the bottom-right corner; otherwise clean with no distortions.

Final Ranking

Rank	Image	Avg Score	Strength	Critical Weakness
1st	Image 1 (Photo)	8	Stunning realism and visual quality	Fails to clearly depict the check
2nd	Image 2 (Digital)	7.2	Best prompt alignment — the check is unmistakable	Flat, non-realistic aesthetic

Key Insight: **Image 1 looks better but Image 2 answers the prompt better.** If accuracy to the original prompt is weighted highest, Image 2 wins. If judged purely on visual merit, Image 1 wins.



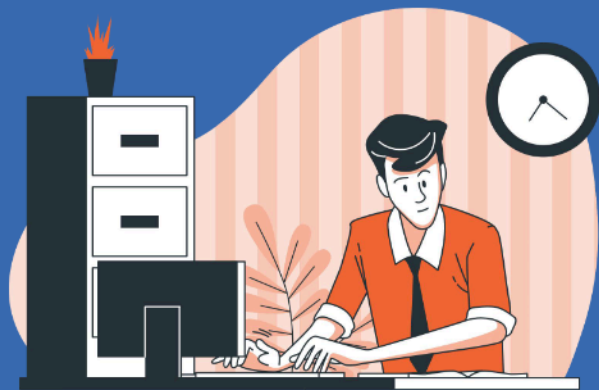
Summary

Summary from All Evaluations

1. **"LLM-as-a-Judge" Methodology:** Utilizing a third-party model (Claude) proved highly effective in providing objective, structured, and consistent evaluations compared to time-consuming manual reviews.
2. **Dominance of Model C (Technical Superiority):** Consistently outperformed others in complex tasks (QA & Requirement Analysis) due to its logical depth and ability to provide concrete examples, such as real JSON bodies.
3. **Model Trade-offs:**
 - Model A (ChatGPT): Excels in instruction following and generative visuals without excessive policy hurdles.
 - Model B (Gemini/Nano Banana): Shows great potential in deep security aspects but is frequently hindered by "safety refusals" on administrative instructions.
4. **The "Hybrid" Ideal:** The best results for industry needs don't come from a single model, but rather a combination of Model C's structure, Model B's security depth, and Model A's execution readiness.
5. **Critical Evaluation Criteria:** Parameters such as Accuracy, Clarity, and Depth are the true differentiators between an AI response that is merely "correct" and one that is "production-ready" for senior/professional levels.

"The goal is not to find the best model, but to understand which model is best for a specific task."

Guided Hands-on



A/B Testing for LLM

Learning Goals

By the end of this session, participants will be able to:

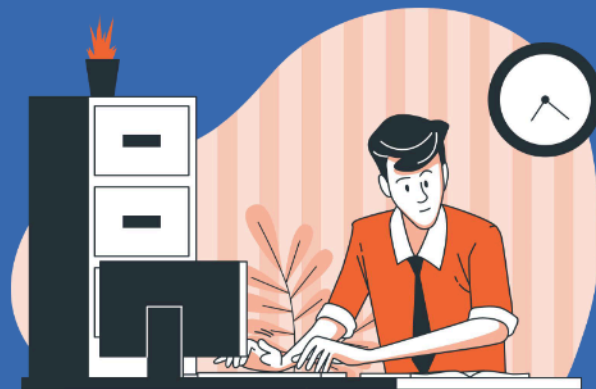
- Compare LLM outputs across models
- Design prompts for evaluation
- Score responses with key criteria
- Use LLM-as-a-Judge for ranking
- Select the right model for the task

Setup

Participants can use:

- Google Colaboratory:
[Session 40 Hands-on_A/B Testing for LLMs.ipynb](#)
- OpenAI API Key
- Gemini API Key

Assignment



A/B Testing for LLM with Use Case

Objective

Enable participants to:

- Design and run an A/B testing experiment using OpenAI and Gemini
- Evaluate LLM outputs using structured criteria (accuracy, clarity, depth, reasoning)
- Apply LLM-as-a-Judge to compare responses objectively
- Identify which model is best suited for a specific use case, not just overall performance

Scenario

You are an AI Engineer tasked with selecting the best LLM for a real-world application.

Your team is building an AI-powered assistant for one of the following domains:

-  Customer Support assistant (handling user complaints, FAQs, and issue resolution)
-  HR assistant (policy explanation, onboarding, or Q&A)
-  Marketing assistant (content generation, campaign ideas)

A/B Testing Requirements

1. Prompt Design

Create 2 prompts:

- Prompt 1 (Structured Task)
Example: generate table, checklist, step-by-step output
- Prompt 2 (Analytical / Creative Task)
Example: reasoning, explanation, idea generation

2. Model Testing

Run both prompts on:

- OpenAI model
- Gemini model

👉 Total outputs = 4 results

3. Evaluation Criteria

Evaluate each output using:

- Accuracy
- Clarity
- Depth
- Reasoning

Score each from 1-10

4. LLM-as-a-Judge

- Use one model as a judge
- Compare outputs (hide model names → Response A vs B)
- Generate:
 - Scores
 - Ranking
 - Short justification

A/B Testing Deliverables

Experiment Results Example

Prompt ID	Task Type	Model	Accuracy	Clarity	Depth	Reasoning	Avg Score	Key Strength	Key Weakness
P1	Structured Task	OpenAI	8	9	8	8	8.25	Well-structured output	Lacks edge cases
P1	Structured Task	Gemini	8	8	9	8	8.25	More detailed scenarios	Slightly less clear
P2	Analytical Task	OpenAI	8	8	7	8	7.75	Clear explanation	Limited depth
P2	Analytical Task	Gemini	9	8	9	9	8.75	Strong reasoning	Slight verbosity



Thank You



Hacktiv8id



Hacktiv8id



Hacktiv8



Hacktiv8 Indonesia